Sprint 3 - S3_05
Busca e Ordenação

Matheus Moreli Durão Silva
Matrícula 12621CBS217
Cibersegurança - Bacharelado - Noturno

2026

Entregável

Implementações e comparação de busca e ordenação. Este documento registra o objetivo do card, a solução desenvolvida e a forma de conferência. Os arquivos de código e as evidências citadas permanecem na mesma pasta deste documento.

O que foi estudado e aplicado

- Busca binária exige dados ordenados
- Bubble, Selection e Insertion Sort
- Análise de complexidade
- Vantagens e limitações

Como conferir

O avaliador pode abrir os arquivos desta pasta, executar os exemplos quando houver código e comparar a saída com a explicação apresentada. Nos cards ligados ao Inventário de Ativos, a solução foi relacionada ao Trabalho 2 sem modificar requisitos já atendidos do CRUD.

Comparação reproduzível

`comparar.py` conta comparações em listas de 20, 100 e 300 itens com ordem crescente, inversa e embaralhada (semente 42). Para 100 itens, Bubble/Selection/Insertion fazem respectivamente 99/4950/99 comparações na entrada ordenada, 4950/4950/4950 na inversa e 4940/4950/2597 na aleatória. A busca binária pelo último item usa 7 comparações após ordenação. Para 300 itens, os valores aleatórios são 44772/44850/24468, e a busca usa 9.

Complexidade e escolha

Bubble com parada antecipada e Insertion são $O(n)$ no melhor caso e $O(n^2)$ no pior; Selection é $O(n^2)$ nos dois. Bubble e Insertion são estáveis, Selection por troca pode não ser. Busca binária custa $O(\log n)$, mas exige a lista previamente ordenada; para uma consulta em lista desordenada, busca linear pode ser melhor. O CRUD principal consulta IDs por dicionário. A tabela completa, os limites e o script reprodutível estão em `relatorio_comparativo.md`.

Conclusão

As contagens resultam de código executado sobre entradas fixas, com as saídas conferidas contra `sorted`.